Android F2FS FIEMAP GC Lab
APK, ODEX, and VDEX Relocation Testing

1 Goal

The goal is to prove that Android package artifacts can keep the same content and inode while their FIEMAP physical extents change due to F2FS garbage collection. The files of interest are normally:

- `base.apk`
- split `*.apk` files
- `base.odex`
- `base.vdex`

The desired outcome is not an application-level rewrite. A successful GC relocation result has these properties:

- the SHA-256 hash before and after is identical;
- the device and inode before and after are identical;
- the FIEMAP physical extents differ;
- F2FS GC counters, especially moved-block counters, increase.

This distinction matters. Game play or settings changes usually mutate app data, caches, shader caches, or save files. They usually do not rewrite `base.apk`, `base.odex`, or `base.vdex`. For these files, the interesting mechanism is F2FS moving still-valid blocks while reclaiming partially invalid sections.

2 Included Files

File	Purpose
`f2fs_fiemap_gc_lab.py`	Host-side Python harness. It drives `adb`, discovers target files, takes before/after snapshots, stages donor files, triggers GC, and writes a summary.
`f2fs_gc_probe.c`	Small native helper. It issues `FS_IOC_FIEMAP`, `F2FS_IOC_GARBAGE_COLLECT`, and `F2FS_IOC_GARBAGE_COLLECT_RANGE`.
`Makefile`	Build targets for a host compile check, Android helper build, and this PDF.
`README.md`	Short command reference.

3 Prerequisites

- A rooted Android device.
- Working `adb`.
- `/data` mounted as F2FS.

- Enough free space on `/data` for donor files.
- Android NDK if you want the recommended native helper.

The helper is strongly recommended. Many Android builds do not ship `filefrag`, and there is no standard shell command for `F2FS_IOC_GARBAGE_COLLECT_RANGE`. Without the helper, the harness can still collect package paths, hashes, stats, and F2FS sysfs counters, but FIEMAP comparison may not be possible.

4 Build the Helper

From the kernel tree:

```
cd tools/f2fs_fiemap_gc_lab
export ANDROID_NDK_HOME=/path/to/android-ndk
make android ANDROID_ARCH=arm64 ANDROID_API=30
```

For a quick local compile check only:

```
make host
```

The Android build produces:

```
tools/f2fs_fiemap_gc_lab/f2fs_gc_probe.android
```

The harness pushes this binary to `/data/local/tmp/f2fs_gc_probe` when `-helper` is supplied.

5 Recommended Workflow: Controlled APK Install

This is the workflow to use when repeatability matters.

5.1 Why this path is better

F2FS segregates data by temperature and writes out of place. Existing package artifacts may be sitting in full, cold, valid sections. GC has little reason to move a fully valid section because it cannot reclaim useful space from it.

The controlled install workflow stages donor files with APK-like extensions, keeps them alive during install, installs the target APK set, then removes the donors before GC. The intended effect is:

1. donor files enter the same cold-data allocation class as APK-like files;
2. package artifacts are written while those donor allocations are present;
3. donor deletion creates invalid blocks near package artifacts;
4. GC now has a partially invalid section that may contain valid package blocks;
5. GC can move those valid package blocks and change FIEMAP.

5.2 Command

For split APKs, repeat `-install-apk`. For one APK, provide one `-install-apk`.

```
python3 f2fs_fiemap_gc_lab.py \
  --package your.package.name \
  --helper ./f2fs_gc_probe.android \
  --install-apk /path/to/base.apk \
  --install-apk /path/to/split_config.apk \
  --range-gc \
  --gc-one-trials 128 \
  --gc-seconds 60 \
  --donor-count 64 \
  --donor-mib 4 \
  --collect-segment-info
```

If you want to remove the existing package first:

```
python3 f2fs_fiemap_gc_lab.py \
  --package your.package.name \
  --helper ./f2fs_gc_probe.android \
  --install-apk /path/to/base.apk \
  --uninstall-first \
  --range-gc
```

6 Existing Play Store Install Workflow

This is lower probability. The harness cannot change how the package artifacts were originally allocated. Donor writes after installation may create general GC pressure, but they may not dirty the sections containing the APK, ODEX, or VDEX files.

```
python3 f2fs_fiemap_gc_lab.py \
  --package your.package.name \
  --helper ./f2fs_gc_probe.android \
  --range-gc \
  --gc-one-trials 128 \
  --gc-seconds 60 \
  --donor-count 64 \
  --donor-mib 4
```

If package discovery misses an artifact, add it explicitly:

```
python3 f2fs_fiemap_gc_lab.py \
  --package your.package.name \
  --helper ./f2fs_gc_probe.android \
  --extra-path /data/app/.../oat/arm64/base.odex \
  --extra-path /data/app/.../oat/arm64/base.vdex \
  --range-gc
```

7 What the Harness Does

7.1 Step 1: Prove eligibility

For every target file, the harness captures:

- `stat` output;
- SHA-256;
- FIEMAP extents through the helper, or `filefrag` if available;
- F2FS sysfs counters.

The result is stored under:

```
snapshots/before.json
snapshots/after.json
summary.txt
summary.json
```

A file is considered relocated only when the content hash and inode remain stable while the FIEMAP extent signature changes.

7.2 Step 2: Manufacture partially invalid sections

The donor-file strategy is controlled by:

```
--donor-dir /data/local/tmp/f2fs_gc_lab_donors
--donor-count 64
--donor-mib 4
--donor-exts apk,vdex,odex
```

Each donor round creates one file per extension. With the values above, the temporary donor footprint is approximately:

```
64 rounds * 4 MiB * 3 extensions = 768 MiB
```

Tune this for the free space on the device. The default is smaller:

```
12 rounds * 8 MiB * 3 extensions = 288 MiB
```

7.3 Step 3: Force ART state before baseline

Unless `-skip-dexopt` is supplied, the harness runs:

```
cmd package compile -m speed -f your.package.name
```

This is done before the baseline snapshot so the test focuses on GC relocation, not normal ART regeneration of ODEX or VDEX files.

7.4 Step 4: Trigger GC

The harness can use three GC paths.

- `-range-gc`: calls the helper's `F2FS_IOC_GARBAGE_COLLECT_RANGE` path for each target extent.
- `-gc-one-trials N`: calls ordinary `F2FS_IOC_GARBAGE_COLLECT` repeatedly through the helper.
- `gc_urgent`: writes F2FS sysfs knobs such as `gc_urgent`, `gc_urgent_sleep_time`, and `gc_remaining_trials`.

The relevant options are:

```
--range-gc
--gc-one-trials 128
--gc-trials 1000
--gc-seconds 60
--urgent-sleep-ms 20
```

7.5 Step 5: Compare and report

The harness writes:

```
summary.txt
summary.json
logs/f2fs_attrs_before.txt
logs/f2fs_attrs_after.txt
logs/helper_gc_range.txt
logs/gc_urgent.txt
```

A successful line in `summary.txt` looks like:

```
MOVED    sha=true  inode=true  extents=3->3 /data/app/.../base.apk
```

That means content and inode stayed stable, but physical extents changed.

8 Interpreting Common Outcomes

8.1 MOVED, hash true, inode true

This is the desired result. The file itself did not change, but FIEMAP changed. This is consistent with F2FS relocating valid blocks during GC.

8.2 MOVED, hash false

This is not a clean GC relocation proof. The file content changed. For ODEX and VDEX this may indicate ART regenerated the artifact. Run dexopt before the baseline snapshot, or use `-skip-dexopt` only when you intentionally want to observe ART changes.

8.3 same-map with GC counters increasing

GC happened, but the target files were not moved. Common reasons:

- their sections were fully valid;
- the sections were not selected as victims;
- donor files landed in hot/warm data while the package artifacts were cold;
- the files or sections were pinned;
- there was not enough GC pressure;
- `-range-gc` was not used.

8.4 same-map with no moved-block counter delta

The GC path probably did not move data blocks. Check:

- `/data` is really F2FS;
- root shell has uid 0;
- helper binary runs on the device architecture;
- `gc_urgent` and moved-block sysfs files exist;
- the filesystem has enough dirty or prefree segments to make GC useful.

9 Segment-Level Debug

Use:

`--collect-segment-info`

This collects:

```
logs/segment_info_before.txt
logs/segment_info_after.txt
logs/f2fs_status_before.txt
logs/f2fs_status_after.txt
```

On kernels with the F2FS proc debug entries enabled, `segment_info` uses this format:

```
segment_type|valid_blocks
segment_type(0:HD, 1:WD, 2:CD, 3:HN, 4:WN, 5:CN)
```

For a normal 4 KiB block, 2 MiB segment F2FS layout, one segment has 512 blocks. If a target's section is full of valid blocks, GC has little reason to move it unless a force-migration path is used. The controlled install workflow tries to make the target section partially invalid by deleting co-located donor files before GC.

10 F2FS Extension Classification

F2FS can classify extensions as cold or hot through:

`/sys/fs/f2fs/<device>/extension_list`

The harness records this in:

```
logs/f2fs_attrs_before.txt
logs/f2fs_attrs_after.txt
```

If `apk`, `odex`, and `vdex` are cold extensions, donor files with those suffixes are more likely to share the package artifact allocation class. If they are not cold extensions, the donor files may not help as much. Changing the extension list is a lab-only intervention because it changes the filesystem policy being tested.

11 Pinning Check

F2FS supports pinned files. Pinned files are intended not to be moved by GC until enough GC failures cause F2FS to drop the guarantee. The harness records:

`gc_pin_file_thresh`

If target package artifacts are pinned, normal GC relocation may be very rare. That is a filesystem policy result, not a harness failure.

12 Safety Notes

- Run this on a lab device or a disposable test image.
- Keep enough free space on `/data`; donor files can be large.
- The harness does not intentionally write target package files.
- The harness does intentionally create and delete files under `/data/local/tmp/f2fs_gc_lab_donors`.
- Aggressive GC can affect device performance during the run.
- If you use `-uninstall-first`, the package is removed before reinstall.

13 Minimal Command Set

Build helper:

```
cd tools/f2fs_fiemap_gc_lab
export ANDROID_NDK_HOME=/path/to/android-ndk
make android ANDROID_ARCH=arm64 ANDROID_API=30
```

Run controlled install:

```
python3 f2fs_fiemap_gc_lab.py \
  --package your.package.name \
  --helper ./f2fs_gc_probe.android \
  --install-apk /path/to/base.apk \
  --range-gc \
  --gc-one-trials 128 \
  --gc-seconds 60 \
  --donor-count 64 \
  --donor-mib 4 \
  --collect-segment-info
```

Inspect:

```
cat f2fs-gc-lab-*/summary.txt
```

14 Expected Reality

A one-in-fifty result is plausible if the app was already installed and only post-install app activity or unrelated random IO was used. The package files must be in victim sections that contain invalid blocks. The controlled install workflow is designed to create that condition. If even the controlled flow does not move `base.apk`, focus next on segment state, extension classification, pinning, and whether the GC range ioctl is actually accepted by the device kernel.